[M6] Skriftligt projektförslag

Vår idé

Ett ekonomispel där man har en simulerad aktiemarknad. Man kommer gå från dag till dag och för varje dag tar man beslut om man köper eller säljer några aktier. Vidare så kommer aktier att ändras från dag till dag. Förändringen kommer vara dynamisk och kommer ske utefter vilken karaktär respektive aktie har.

Målet är att man efter ett givet antal dagar ska ha så mycket pengar som möjligt.

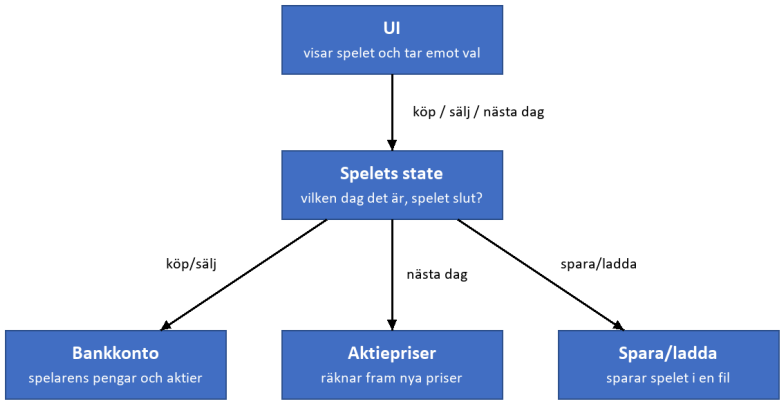
Systemets delar

Systemet består av fem delar:

- UI: visar spelet för spelaren och tar emot vad spelaren vill göra.
- Spelets state: håller koll på vilken dag det är och när spelet är slut.
- Bankkonto: sparar hur mycket pengar spelaren har och vilka aktier spelaren äger.
- Aktiepriser: räknar fram nya priser på aktierna varje dag, utifrån varje akties karaktär.
- Spara/ladda: sparar spelet i en fil så att man kan fortsätta senare.

Spelaren väljer i UI om den vill köpa, sälja eller gå till nästa dag. Vid köp eller sälj kontrolleras att pengarna räcker, och sedan uppdateras bankkontot. Vid nästa dag räknas nya aktiepriser fram och dagen räknas upp. När sista dagen är nådd är spelet slut, och resultatet är pengarna plus värdet av aktierna.

Systemets delar



Representation

Spelarens mängd pengar på kontot kan representeras som en integer. Även priserna på aktier bör kunna representeras som integer. För olika viktningar när förändringar av aktiepris ska generas, kanske float är mer passande.

För att lagra värden för respektive aktie skulle dictionary passa.

Komplexitet och förenkling

Vi börjar med en förenklad version och bygger på delar för att göra systemet mer avancerat. Den mest förenklade versionen ska vara textbaserad simulera en enda aktie vars pris ändras slumpmässigt per tidsenhet. Spelaren bör också vid denna version kunna köpa och sälja aktien och gå vidare till nästa dag.

Nästa utveckling är att lägga till flera aktier och ge dem olika karaktär, där olika egenskaper finns hos varje aktie, till exempel en väldigt volatil eller en mer stabil. Systemet ska också kunna avgöra om spelaren har tillräckligt med pengar för sitt köp.

Den sista utvecklingen av systemet ska vara att spelet kan sparas till en fil och även laddas in vid nästa start. Det mest praktiska formatet att spara i är JSON. Det ska även gå att se prishistoriken för varje aktie.

Om tid finns över och allt fungerar bra så kan en grafiskt version utvecklas också där man antingen använder bibliotek som “Pygame” eller “PyQt5” för att skapa ett snyggt UI med prisgrafer och interaktiva knappar. Det går även att köra programmet som webbserver med biblioteket “Flask” och designa UI i HTML och CSS.

Personliga utvecklingsmål

För Eriks utvecklingsmål om att hantera stora mängder data på effektivt sätt, kan vi undersöka om logiken kring aktiernas förändring kan baseras på mycket data eller liknande.

Josephs utvecklingsmål är att lära sig mer om python-bibliotek och filhantering. Detta kommer att vara nyttigt när det kommer till sparfunktionen i spelet och även om det ska fungera grafiskt senare.

Arbetsfördelning och planering

1. Arbetsfördelning

Erik – Aktiemarknad (`market.py`)

- Skapa och uppdatera aktiepriser.
- Hantera olika aktietyper och volatilitet.
- Uppdatera aktiernas prishistorik.
- Ansvara för en större del av integrationen och testningen av spelet.

Joseph – Handel och sparfunktion (`portfolio.py` och `save_load.py`)

- Implementera köp och försäljning av aktier.
- Hålla reda på spelarens pengar och aktieinnehav.
- Implementera sparning och inläsning med JSON.

Gemensamt – Huvudprogram och testning (`main.py`)

- Bestämma hur programmets delar ska kopplas ihop.
- Skapa spelets huvudloop och hantera övergången till nästa dag.
- Koppla ihop modulerna och kontrollera att de fungerar tillsammans.
- Hjälpas åt med felsökning och slutlig testning.

2. Arbetsordning

1. **Planering:** Bestäm hur spelets data ska struktureras och hur funktionerna ska kommunicera.
2. **Utveckling:** Erik utvecklar aktiemarknaden och Joseph utvecklar handelssystemet. Delarna testas separat.
3. **Integration:** Vi kopplar ihop modulerna och testar att de fungerar tillsammans.
4. **Sparfunktion:** Joseph implementerar sparning och inläsning när datastrukturen är bestämd.
5. **Sluttestning:** Vi testar hela spelet, rättar fel och förbättrar programmet.

3. Samarbete

Vi använder Git för att dela kod och hålla ordning på ändringar. Vi bestämmer gemensamma mål för varje arbetspass och går igenom resultatet innan vi fortsätter.

För att arbetsbördan ska bli jämn ansvarar Erik för en större del av integrationen och testningen, medan Joseph fokuserar på handelssystemet och sparfunktionen. Båda hjälper till med huvudprogrammet och ser till att förstå hur hela spelet fungerar.